

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

CASO DE ESTUDIO A DESARROLLAR POR LOS GRUPOS DE ESTUDIANTES

Usted ha sido contratado como **ingeniero de datos senior** para recuperar, procesar y disponibilizar datos históricos de viajes almacenados en archivos **Parquet**. La organización está migrando su arquitectura analítica hacia un modelo tipo **Data Lakehouse**, pero durante la auditoría inicial se detectaron archivos con esquemas inconsistentes, datos incompletos, registros inválidos y archivos potencialmente corruptos.

Para esta evaluación deberá utilizar datos reales publicados por **NYC Taxi & Limousine Commission**, disponibles en formato Parquet desde su portal oficial. También deberá incorporar archivos problemáticos del repositorio de pruebas de **Apache Parquet**, con el fin de demostrar que su solución puede detectar, clasificar y aislar archivos dañados sin detener el pipeline completo.

Su tarea consiste en diseñar e implementar un pipeline ETL avanzado utilizando **Apache Spark** o **PySpark**. El pipeline deberá extraer los archivos Parquet desde el repositorio origen, organizarlos en una arquitectura por capas, diagnosticar sus esquemas, detectar archivos corruptos o inconsistentes, reconstruir un esquema canónico común, transformar los datos, aplicar reglas de calidad, separar registros válidos e inválidos, generar métricas de auditoría y cargar el resultado final en una base de datos consultable.

El proceso deberá contemplar, como mínimo, las capas **raw**, **bronze**, **silver**, **gold**, **quarantine** y **audit**. En la capa raw se conservarán los archivos originales sin modificación. En bronze se almacenarán los datos leídos y reconstruidos. En silver se ubicarán los datos limpios, normalizados y validados. En gold se cargarán las tablas analíticas finales. En quarantine deberán almacenarse las referencias a archivos o registros rechazados, junto con la causa técnica o de negocio. En audit deberán registrarse las métricas del procesamiento.

El alumno deberá descargar al menos los archivos Parquet correspondientes a viajes de taxi amarillo, taxi verde y vehículos FHVHV para varios meses del año 2023. Además, deberá incorporar archivos problemáticos desde el repositorio Apache Parquet Testing. El objetivo es que el pipeline sea capaz de procesar fuentes heterogéneas, manejar diferencias reales de esquema y responder correctamente ante archivos dañados o mal estructurados.

La solución deberá generar un inventario técnico de archivos que incluya nombre, ruta, tamaño, partición, número de registros, número de columnas, estado de lectura, hash de esquema, mensaje de error y fecha de procesamiento. También deberá generar una tabla de registros rechazados y una tabla resumen de métricas de calidad por servicio, año y mes.

El dataset final deberá unificar los distintos tipos de servicio bajo un esquema canónico común. Este esquema deberá incluir, al menos, identificador del viaje, tipo de servicio, fechas de recogida y llegada, duración del viaje, distancia, zonas de origen y destino, tipo de pago, tarifa, propina, total pagado, año, mes, archivo fuente, fecha de ingesta y estado de calidad.

Durante la transformación, el alumno deberá calcular métricas derivadas como duración del viaje, velocidad promedio, tarifa por milla, porcentaje de propina e indicador de viaje sospechoso. También deberá detectar y rechazar registros con fechas inválidas, montos negativos, distancia igual o menor que cero, duración inválida, velocidad irreal, duplicados técnicos y valores nulos en campos críticos.

El resultado final deberá cargarse en una base de datos. Se aceptan PostgreSQL, MySQL, SQL Server, DuckDB, SQLite, Delta Lake, Hive Metastore, BigQuery, Snowflake o Redshift. La entrega

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

deberá incluir las tablas `gold_trips_clean`, `gold_daily_revenue`, `gold_location_performance`, `quality_rejected_records`, `quality_metrics_summary` y `audit_file_inventory`.

Además del código, el alumno deberá entregar un informe técnico profesional donde explique la arquitectura implementada, la estrategia de extracción, el diagnóstico de los archivos, la metodología de reconstrucción, las reglas de calidad aplicadas, las decisiones de optimización Spark, los resultados obtenidos, las limitaciones del proceso y las conclusiones finales.

La evaluación valorará especialmente la capacidad del alumno para tomar decisiones técnicas justificadas. No se aceptará como solución válida ignorar archivos dañados sin trazabilidad, convertir todo el procesamiento a Pandas, eliminar registros sin explicar el motivo o cargar datos finales sin validación previa.

Al finalizar, el alumno deberá demostrar mediante consultas SQL que los datos fueron correctamente cargados y que las tablas finales pueden ser utilizadas para análisis. También deberá presentar una reflexión crítica sobre los riesgos de procesar datos dañados, el impacto de la calidad de datos en la toma de decisiones y las medidas que implementaría para evitar que este problema vuelva a ocurrir en un ambiente productivo.

Condición adicional:

El pipeline deberá poder ejecutarse más de una vez sin duplicar resultados, deberá aceptar parámetros externos mediante un archivo de configuración y deberá generar un `process_id` único por ejecución. Además, el alumno deberá demostrar que su proceso es idempotente, auditable y escalable.

1. Fuente oficial de datos

Para esta evaluación se utilizará como fuente base el dataset público de **NYC Taxi & Limousine Commission Trip Record Data**, que publica archivos históricos de viajes en formato **Parquet**. La página oficial indica que, debido al tamaño de los datasets, los archivos de viajes se almacenan en formato Parquet.

Repositorio principal de descarga: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page?utm_source=chatgpt.com

El alumno deberá descargar archivos Parquet desde la página oficial:

NYC TLC Trip Record Data: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page?utm_source=chatgpt.com

También puede consultarse la ficha del dataset en el **Registry of Open Data on AWS**, donde se describe como un conjunto de datos de viajes de taxis y vehículos de alquiler en Nueva York. El acceso directo por S3 puede requerir cuenta AWS, pero la propia ficha indica que existen descargas anónimas desde la página documental del dataset.

https://registry.opendata.aws/nyc-tlc-trip-records-pds/?utm_source=chatgpt.com

2. Archivos mínimos que debe descargar el alumno

El alumno deberá descargar, como mínimo, los siguientes archivos Parquet desde la fuente oficial:

`yellow_tripdata_2023-01.parquet`
`yellow_tripdata_2023-02.parquet`
`yellow_tripdata_2023-03.parquet`

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

yellow_tripdata_2023-04.parquet
green_tripdata_2023-01.parquet
green_tripdata_2023-02.parquet
fhvvhv_tripdata_2023-01.parquet
yellow_tripdata_2022-01.parquet
yellow_tripdata_2022-02.parquet
yellow_tripdata_2021-01.parquet
yellow_tripdata_2020-01.parquet

3. Fuente complementaria para archivos Parquet dañados

Además de los datos reales de viajes, el alumno deberá analizar archivos Parquet problemáticos provenientes del repositorio oficial de pruebas de Apache Parquet:

Apache Parquet Testing Repository : https://github.com/apache/parquet-testing?utm_source=chatgpt.com

Este repositorio contiene archivos de prueba, incluyendo una carpeta de datos problemáticos o “bad data” utilizados para reproducir errores en archivos Parquet.

https://github.com/apache/parquet-testing?utm_source=chatgpt.com

Deberá usar estos archivos como insumo para demostrar que su pipeline puede:

- Detectar archivos ilegibles.
- Capturar excepciones de lectura.
- Separar archivos corruptos.
- Registrar evidencia técnica del fallo.
- Evitar que un archivo dañado detenga todo el proceso ETL.
- Documentar qué se pudo recuperar y qué no.

4. Preparación obligatoria del repositorio de trabajo

El alumno deberá construir la siguiente estructura local o en un entorno distribuido:

```
etl_spark_parquet_advanced/  
|  
|├─ data/  
| |├─ raw/  
| | |├─ yellow/  
| | | |├─ year=2023/month=01/yellow_tripdata_2023-01.parquet  
| | | |├─ year=2023/month=02/yellow_tripdata_2023-02.parquet  
| | | |├─ year=2023/month=03/yellow_tripdata_2023-03.parquet  
| | | |└─ year=2023/month=04/yellow_tripdata_2023-04.parquet  
| | |  
| | |├─ green/  
| | | |├─ year=2023/month=01/green_tripdata_2023-01.parquet  
| | | |└─ year=2023/month=02/green_tripdata_2023-02.parquet  
| | |  
| | |├─ fhvvhv/  
| | | |└─ year=2023/month=01/fhvvhv_tripdata_2023-01.parquet  
| | |  
| | |└─ bad_parquet/  
| | | |└─ archivos_provenientes_de_apache_parquet_testing/
```

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

```
| |
| |├── bronze/
| |├── silver/
| |├── gold/
| |├── quarantine/
| |└── audit/
| |
| ├── metadata/
| |├── expected_schema_yellow.json
| |├── expected_schema_green.json
| |├── expected_schema_fhvhv.json
| |├── canonical_schema_trips.json
| |└── business_rules.json
| |
| ├── notebooks/
| |├── 01_extraccion.ipynb
| |├── 02_diagnostico_reconstruccion.ipynb
| |├── 03_transformacion_validacion.ipynb
| |├── 04_carga_base_datos.ipynb
| |└── 05_reporte_calidad_conclusiones.ipynb
| |
| ├── src/
| |├── extract.py
| |├── schema_recovery.py
| |├── transformations.py
| |├── quality_rules.py
| |├── load.py
| |└── utils.py
| |
| ├── config/
| |└── etl_config.yaml
| |
| └── README.md
```

5. Problema técnico a resolver

Durante una auditoría inicial, se detectaron los siguientes problemas:

1. Algunos archivos Parquet no pueden leerse correctamente.
2. Existen diferencias de esquema entre archivos yellow, green y fhvhv.
3. Algunas columnas representan el mismo concepto, pero tienen nombres distintos.
4. Existen columnas presentes en un tipo de servicio, pero ausentes en otro.
5. Hay valores nulos en campos críticos.
6. Existen fechas inválidas o inconsistentes.
7. Algunos registros tienen importes negativos.
8. Hay viajes con distancia igual a cero.
9. Existen posibles duplicados operacionales.
10. Algunos archivos deben ser enviados a cuarentena.
11. El dataset final debe quedar unificado bajo un esquema canónico.
12. El resultado debe cargarse en una base de datos consultable.

El alumno deberá construir una solución que no dependa de supuestos manuales frágiles, sino de reglas explícitas, validaciones técnicas, logs y métricas.

Metodología de Trabajo

Fase 1: Extracción de datos

Objetivo de la fase

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

Extraer los archivos Parquet desde el repositorio origen, organizar los datos en la zona **raw** y construir un inventario técnico inicial.

Actividades obligatorias

El alumno deberá:

1. Descargar los archivos Parquet indicados.
2. Almacenar los archivos sin modificación en la capa raw.
3. Leer cada archivo de forma individual con Spark.
4. Leer carpetas particionadas completas.
5. Identificar el tipo de servicio: yellow, green o fhvhv.
6. Extraer metadatos técnicos por archivo.
7. Detectar archivos vacíos, ilegibles o corruptos.
8. Registrar errores de lectura.
9. Crear una tabla de inventario de archivos.

Inventario mínimo requerido

El alumno deberá generar una tabla llamada: `naudit_file_inventory`

Con los siguientes campos:

```
process_id
source_system
service_type
file_name
file_path
file_size_mb
partition_year
partition_month
read_status
record_count
column_count
schema_hash
error_message
processed_at
```

Criterio de aceptación

La fase será considerada correcta solo si el alumno demuestra que puede leer archivos válidos y aislar archivos problemáticos sin que el proceso completo falle.

Fase 2: Diagnóstico y reconstrucción de archivos Parquet

Objetivo de la fase

Diagnosticar inconsistencias estructurales, recuperar la mayor cantidad posible de información útil y reconstruir un dataset operativo bajo un esquema canónico.

Actividades obligatorias

El alumno deberá:

1. Comparar el esquema real de cada archivo contra un esquema esperado.
2. Detectar columnas faltantes.
3. Detectar columnas adicionales no documentadas.
4. Detectar columnas con tipos incompatibles.
5. Detectar columnas equivalentes con nombres distintos.
6. Crear un esquema canónico para todos los viajes.
7. Homologar columnas entre yellow, green y fhvhv.

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

8. Agregar columnas ausentes con valores nulos controlados.
9. Convertir tipos de datos con manejo de errores.
10. Enviar archivos no recuperables a cuarentena.
11. Documentar el motivo de cada exclusión.

Ejemplo de esquema canónico mínimo

El alumno puede enriquecerlo, pero no reducirlo:

trip_id
service_type
vendor_id
pickup_datetime
dropoff_datetime
passenger_count
trip_distance
pickup_location_id
dropoff_location_id
payment_type
fare_amount
extra_amount
mta_tax
tip_amount
tolls_amount
total_amount
congestion_surcharge
airport_fee
year
month
source_file
ingestion_timestamp
quality_status

Reglas de homologación sugeridas

El alumno deberá construir una matriz como esta:

Concepto canónico	Yellow taxi	Green taxi	FHVHV
pickup_datetime	tpep_pickup_datetime	lpep_pickup_datetime	pickup_datetime
dropoff_datetime	tpep_dropoff_datetime	lpep_dropoff_datetime	dropoff_datetime
pickup_location_id	PULocationID	PULocationID	PULocationID
dropoff_location_id	DOLocationID	DOLocationID	DOLocationID
total_amount	total_amount	total_amount	base_passenger_fare + tolls + sales_tax
service_type	yellow	green	fhvhv

El alumno deberá justificar todas las reglas de homologación que aplique.

Fase 3: Reconstrucción ante archivos dañados

Objetivo de la fase

Diseñar una estrategia profesional para tratar archivos dañados o parcialmente ilegibles.

Reglas obligatorias

El alumno deberá clasificar cada archivo problemático en una de estas categorías:

RECUPERABLE_SCHEMA_MISMATCH

RECUPERABLE_MISSING_COLUMNS

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

RECUPERABLE_TYPE_CASTING
PARTIALLY_RECOVERABLE
NOT_RECOVERABLE_CORRUPT_METADATA
NOT_RECOVERABLE_EMPTY_FILE
NOT_RECOVERABLE_UNSUPPORTED_FORMAT

Cuarentena obligatoria

Los archivos o registros inválidos deberán enviarse a: data/quarantine/

La cuarentena debe incluir:

Archivo original o referencia al archivo.

Motivo de rechazo.

Excepción capturada.

Fecha de procesamiento.

Etapas donde falló.

Acción recomendada.

Importante

No se calificará positivamente una solución que simplemente ignore archivos corruptos sin registrar evidencia. El alumno debe demostrar que sabe diferenciar entre:

Datos recuperables.

Datos parcialmente recuperables.

Datos no recuperables.

Errores de esquema.

Errores físicos de archivo.

Errores de calidad de negocio.

Fase 4: Transformación de datos

Objetivo de la fase

Convertir el dataset reconstruido en información limpia, consistente y analíticamente útil.

Transformaciones obligatorias

El alumno deberá implementar al menos las siguientes transformaciones:

1. Normalización de nombres de columnas.
2. Conversión de fechas a timestamp.
3. Cálculo de duración del viaje en minutos.
4. Cálculo de velocidad promedio estimada.
5. Normalización de importes monetarios.
6. Conversión de valores negativos inválidos a registros rechazados.
7. Generación de identificador único trip_id.
8. Eliminación de duplicados técnicos.
9. Enriquecimiento con campos de partición year y month.
10. Creación de campo service_type.
11. Creación de campo source_file.
12. Creación de campo ingestion_timestamp.
13. Creación de campo quality_status.

Campos derivados mínimos

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

El dataset transformado deberá incluir:

trip_duration_minutes
average_speed_mph
fare_per_mile
tip_percentage
is_airport_trip
is_suspicious_trip
processing_date

Ejemplo de reglas avanzadas

El alumno deberá marcar como sospechoso un viaje cuando ocurra una o más de las siguientes condiciones:

trip_distance <= 0
total_amount <= 0
fare_amount < 0
trip_duration_minutes <= 0
trip_duration_minutes > 480
average_speed_mph > 100
tip_percentage > 100
pickup_datetime > dropoff_datetime
pickup_datetime está en el futuro

Fase 5: Validación de calidad de datos

Objetivo de la fase

Medir y demostrar la calidad del dataset antes de cargarlo en la base de datos.

Reglas de calidad mínimas

El alumno deberá implementar validaciones para:

Nulidad de campos críticos.
Rangos válidos de fechas.
Rangos válidos de montos.
Rangos válidos de distancia.
Rangos válidos de duración.
Duplicados.
Integridad de particiones.
Consistencia entre año/mes de partición y fecha real del viaje.
Porcentaje de registros rechazados.
Distribución de registros por servicio.
Detección de outliers.

Tabla obligatoria de registros rechazados

Debe crearse una tabla o archivo llamado: quality_rejected_records

Con los campos:

process_id
trip_id
service_type
source_file

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

rejection_stage
rejection_rule
rejection_column
original_value
technical_reason
business_reason
rejected_at

Tabla obligatoria de métricas de calidad

Debe crearse una tabla llamada: quality_metrics_summary

Con estos campos:

process_id
service_type
year
month
total_records
valid_records
rejected_records
duplicate_records
null_critical_records
suspicious_records
quality_percentage
processed_at

Fase 6: Enriquecimiento analítico

Objetivo de la fase

Preparar una capa **gold** orientada a análisis de negocio y explotación posterior.

Agregaciones obligatorias

El alumno deberá construir al menos tres tablas analíticas:

Tabla gold_trips_clean

Tabla granular de viajes limpios:

trip_id
service_type
pickup_datetime
dropoff_datetime
trip_duration_minutes
trip_distance
pickup_location_id
dropoff_location_id
payment_type
fare_amount
tip_amount
total_amount
tip_percentage
average_speed_mph
year
month
source_file

Tabla gold_daily_revenue

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

Resumen diario:

service_type
trip_date
total_trips
total_revenue
average_fare
average_tip
average_trip_distance
average_trip_duration
rejected_records
quality_percentage

Tabla gold_location_performance

Resumen por zona de origen y destino:

service_type
pickup_location_id
dropoff_location_id
total_trips
total_revenue
average_fare
average_distance
average_duration
suspicious_trip_count

Fase 7: Carga en base de datos

Objetivo de la fase

Persistir los datos finales en una base de datos consultable y demostrar que la carga fue exitosa mediante consultas SQL.

Bases de datos permitidas

El alumno deberá usar una de las siguientes opciones:

PostgreSQL
MySQL
MariaDB
SQLite
Delta Lake
Hive Metastore
BigQuery
Snowflake
Redshift

Tablas obligatorias en destino

El alumno deberá cargar como mínimo:

gold_trips_clean
gold_daily_revenue
gold_location_performance
quality_rejected_records
quality_metrics_summary
audit_file_inventory

Consultas de verificación obligatorias

El alumno deberá entregar resultados de las siguientes consultas:

SELECT
 service_type,

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

```
COUNT(*) AS total_trips,  
SUM(total_amount) AS total_revenue  
FROM gold_trips_clean  
GROUP BY service_type  
ORDER BY total_revenue DESC;
```

```
SELECT  
    service_type,  
    year,  
    month,  
    total_records,  
    valid_records,  
    rejected_records,  
    quality_percentage  
FROM quality_metrics_summary  
ORDER BY year, month, service_type;
```

```
SELECT  
    pickup_location_id,  
    dropoff_location_id,  
    COUNT(*) AS total_trips,  
    SUM(total_amount) AS total_revenue,  
    AVG(trip_duration_minutes) AS avg_duration  
FROM gold_trips_clean  
GROUP BY pickup_location_id, dropoff_location_id  
ORDER BY total_revenue DESC  
LIMIT 20;
```

Requisitos obligatorios

Para considerar la evaluación, el alumno deberá cumplir con los siguientes requisitos:

- Uso obligatorio de Apache Spark o PySpark.
- Lectura real de archivos Parquet.
- Manejo de archivos corruptos o problemáticos.
- Comparación de esquemas.
- Reconstrucción de esquema canónico.
- Unificación de múltiples fuentes Parquet.
- Separación de capas raw, bronze, silver y gold.
- Generación de logs.
- Generación de métricas de calidad.
- Carga en base de datos.
- Consultas SQL de validación.
- Documentación técnica profesional.
- Reflexión crítica final.

Requisitos de optimización Spark

El alumno deberá justificar e implementar al menos cinco de las siguientes técnicas:

- Uso de inferSchema controlado o esquema explícito.
- Lectura selectiva de columnas.
- Predicate pushdown.
- Partition pruning.
- Manejo del problema de archivos pequeños.
- Escritura particionada.

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

Control del número de archivos de salida.
Uso de explain() para analizar planes físicos.
Entre otros..

Entregables obligatorios

El alumno deberá entregar:

Código fuente

Puede entregarse como:
Notebook Jupyter
Repositorio Git

Archivo README.md

Debe explicar:
Cómo descargar los datos.
Cómo preparar el entorno.
Cómo ejecutar el pipeline.
Qué parámetros deben configurarse.
Qué base de datos se utilizó.
Cómo validar los resultados.

Documento técnico

Debe incluir:
Descripción del caso.
Fuente de datos utilizada.
Arquitectura propuesta.
Diagrama del flujo ETL.
Descripción de cada fase.
Problemas encontrados.
Estrategia de reconstrucción.
Reglas de calidad.
Decisiones de optimización.
Resultados obtenidos.
Limitaciones.
Conclusiones.
Reflexión personal.

Restricciones técnicas

No se admitirán trabajos si:

Procesa únicamente archivos CSV.
Convierte todo a Pandas para resolver el problema.
Ignora archivos corruptos sin registrarlos.
No genera tabla de auditoría.
No separa registros válidos e inválidos.
No carga los resultados en una base de datos.
No explica sus decisiones técnicas.
No entrega conclusiones.

El uso de Pandas solo se permitirá para inspecciones auxiliares pequeñas, nunca como motor principal de procesamiento.

Criterios de evaluación

PROYECTO II PARCIAL – MODELADO AVANZADO DE BASE DE DATOS

Criterio	Puntaje
Descarga, organización y extracción de archivos Parquet reales	10
Inventario técnico y auditoría de archivos	10
Diagnóstico de archivos dañados o inconsistentes	15
Reconstrucción de esquema canónico	15
Transformaciones avanzadas con Spark	15
Reglas de calidad y separación de rechazados	10
Carga correcta en base de datos	10
Optimización Spark y justificación técnica	5
Documentación profesional	5
Reflexiones y conclusiones	5
Total	100